

Apunte de Clase: Ensamblado, Vinculación, Carga y Ejecución

Introducción

Una vez que el compilador ha traducido el código fuente a lenguaje ensamblador, todavía faltan varias etapas para obtener un programa ejecutable. Estas etapas son:

1. Ensamblado
2. Vinculación (Linkeo)
3. Carga y Ejecución

Cada una de estas fases resuelve problemas específicos, como la resolución de referencias a etiquetas, la unión de múltiples módulos y la preparación final para la ejecución en memoria.

1. Ensamblado

1.1 Objetivo principal

El ensamblador convierte el código ensamblador (.asm) en código máquina (archivo objeto .obj u .o). Para ello, debe traducir cada instrucción mnemotécnica (ej: MOV, ADD, JMP) a su representación binaria.

1.2 El problema de la "forward reference"

Una dificultad clásica del ensamblado es la referencia hacia adelante. Esto ocurre cuando una instrucción hace referencia a una etiqueta que aún no ha sido definida en el código fuente.

En la primera pasada, el ensamblador no sabe qué dirección tiene miEtiqueta. Para resolver esto, los ensambladores utilizan un proceso de dos pasadas.

1.3 Ensamblado de dos pasadas

Primera pasada

- Función principal: Construir la tabla de símbolos.
- ILC (Instruction Length Counter): Es un contador que lleva la posición (dirección) de cada instrucción dentro del código. Se incrementa según la longitud de cada instrucción.
- Al finalizar la primera pasada, el ensamblador conoce la dirección de todas las etiquetas.

Ejemplo de tabla generada en primera pasada:

Etiqueta	Valor (ILC)
MARIA	100

ROBERTA	111
MARILYN	125
STEPHANY	129

Además de la tabla de símbolos, se construyen:

- Tabla de opcodes: relaciona cada mnemónico con su código máquina.
- Tabla de pseudoinstrucciones: para directivas como EQU, DB, DW, etc.

Segunda pasada

- Se vuelve a leer el código línea por línea.
- Se traduce cada instrucción a código máquina.
- Cuando aparece una etiqueta, se reemplaza por su valor de memoria obtenido de la tabla de símbolos.
- Se genera finalmente el archivo objeto.

1.4 Errores detectados en el ensamblado

La segunda pasada también es responsable de detectar errores como:

- Símbolo usado pero no definido.
- Símbolo definido más de una vez (duplicado).
- Opcode inválido (no existe en la tabla de opcodes).
- Cantidad incorrecta de operandos (faltan o sobran).
- Uso ilegal de registros (ej: jmp eax).
- Ausencia de la sentencia END.

2. Tabla de Símbolos e Implementación Eficiente

2.1 ¿Qué es la tabla de símbolos?

Es una estructura clave → valor donde la clave es el nombre del símbolo (etiqueta) y el valor es su dirección de memoria.

2.2 Implementación ingenua: arreglo de tuplas

Una forma sencilla es usar un arreglo donde cada elemento es un par (nombre, valor). La búsqueda es $O(n)$, lo cual es ineficiente para programas grandes.

2.3 Mejora: Hash Table

- Se convierte el nombre del símbolo a un número (ej: sumando valores ASCII).
- Se aplica una función hash: $\text{hash}(\text{nombre}) \% K$ (donde K es el tamaño de la tabla).
- Si K es cercano a la cantidad de símbolos (n), la búsqueda promedio es $O(1)$.

Ejemplo:

text

"Andy" $\rightarrow (41 * 110 * 100 * 121) \% 8 \rightarrow 54571000 \% 8 \rightarrow 0$

De esta forma, el símbolo "Andy" se almacena en la posición 0 de la tabla hash.

3. Vinculación (Linkeo)

3.1 ¿Por qué es necesaria?

Cuando un programa se divide en múltiples módulos (archivos fuente separados), cada módulo se ensambla individualmente. Todos ellos asumen que su dirección de inicio es 0 (espacio de direcciones virtuales separado). El linker se encarga de:

1. Unir todos los módulos en un solo espacio de direcciones.
2. Resolver las referencias entre módulos (funciones o variables públicas).
3. Reubicar (relocalizar) las direcciones según la posición final de cada módulo.

3.2 Pasos de la vinculación

1. Construir una tabla de todos los módulos y sus longitudes.
2. Asignar una dirección de inicio a cada módulo (por ejemplo: módulo A empieza en 0, módulo B en 400, etc.).
3. Sumar un offset (dirección de inicio) a todas las instrucciones que referencian memoria dentro del módulo.
4. Resolver referencias externas: buscar en otros módulos las funciones o variables que se usan y sustituir sus direcciones finales.

Ejemplo:

Módulo	Longitud	Dirección de inicio
A	400	0
B	600	400

3.3 Estructura de un módulo objeto

Un archivo objeto (.obj) contiene varias secciones necesarias para el linker:

1. Identificación: nombre del módulo.
2. Entry Point Table (EPT): lista de símbolos públicos (exportados) que pueden ser usados por otros módulos, junto con su dirección.
3. External Reference Table (ERT): lista de símbolos que este módulo necesita pero que pertenecen a otros módulos.
4. Código reubicable: instrucciones que dependen de la dirección de carga y deben ser ajustadas.
5. Checksum: para verificar integridad.

3.4 Vinculación Dinámica

En lugar de linkear todo al momento de generar el ejecutable, se puede hacer durante la carga o ejecución.

- Precarga: la librería se carga al inicio del programa.
- A demanda: se carga solo cuando se llama por primera vez a una función de la librería.

Ventajas:

- Varios programas pueden compartir una misma librería en memoria.
- Reduce el tamaño de los ejecutables.

Desventajas:

- Genera dependencias externas (si la librería falta o cambia, el programa puede fallar).

4. Carga y Ejecución

4.1 Función del cargador (loader)

Una vez que el linker ha producido un ejecutable, el cargador del sistema operativo:

- Copia las instrucciones y datos del ejecutable a la memoria RAM.
- Transfiere el control al punto de entrada del programa.

4.2 Problema de memoria insuficiente

Si la memoria RAM es limitada, el sistema operativo puede usar paginación o segmentación para mover partes del programa entre RAM y disco. Gracias a la memoria virtual, las direcciones dentro del programa no cambian, aunque físicamente estén en distintas páginas.

- El cargador no necesita modificar el código si hay paginación.
- La tabla de páginas del SO se encarga de la traducción de direcciones virtuales a físicas.

5. Resumen de errores según etapa

Etapa	Tipo de error que detecta
Ensamblado (1° pasada)	Forward reference (se resuelve en 2° pasada)
Ensamblado (2° pasada)	Símbolo no definido, duplicado, opcode inválido, operandos incorrectos, falta END

Vinculación	Referencia externa no resuelta, solapamiento de módulos
Carga	Memoria insuficiente, ejecutable corrupto

Las etapas de ensamblado, vinculación, carga y ejecución son fundamentales para transformar un programa en lenguaje ensamblador en un proceso en ejecución dentro de la memoria de la computadora. El uso de dos pasadas en el ensamblado resuelve el problema de referencias hacia adelante, mientras que la tabla de símbolos y la vinculación permiten trabajar con módulos independientes. Finalmente, el cargador se encarga de ubicar el programa en memoria, aprovechando mecanismos como la paginación para manejar recursos limitados.